

次世代AIエージェントアーキテクチャ: 確率的生成から証明可能な自律性へ

2025年版 AET-OS構築に向けた技術調査および戦略的実装レポート

1. エグゼクティブサマリー: 不確実性の克服と「証明可能な自律性」への転換

貴殿が現在開発を進めている「AET-OS (Agentic Evolutionary Technology - Operating System)」は、単なるタスク自動化ツールではなく、コード修正やシステム設計というミッションクリティカルな領域を担う基盤システムであると認識している。この野心的な目標において、現在(2024年後半から2025年初頭)のAIエージェント研究が直面している最大の課題は、大規模言語モデル(LLM)の本質的な特性である「確率的な曖昧さ」をいかにして工学的な「確実性」へと昇華させるかという点に集約される。

従来のLLM中心のパラダイムは、ReActやChain of Thought (CoT)といったプロンプティング技術によって一定の成果を上げてきたが、これらは本質的に確率論的であり、複雑なシステム設計において許容できない「幻覚(Hallucination)」や、停止条件を見失う「無限ループ」といった信頼性の欠如を露呈してきた。これに対し、2025年の最先端(State-of-the-Art: SOTA)トレンドは、この確率的な性質を、形式検証(**Formal Verification**)、動的オーケストレーション(**Dynamic Orchestration**)、そして**強化学習による自己修正(RL-based Self-Correction)**という三つの柱によって制御し、「証明可能な自律性(Provable Autonomy)」を実現する方向へと急速にシフトしている。

本レポートは、Microsoft ResearchのAutoGen v0.4におけるアーキテクチャの刷新、DeepMindによる自己修正メカニズムSCoRe、そして形式手法をエージェントループに統合するVeCoGenやQWEDプロトコルといった最新の研究成果を網羅的に調査・分析したものである。特に、貴殿が関心を寄せる「堅牢性」「形式検証」「構造化」の3点に焦点を当て、単なる文献レビューに留まらず、AET-OSの実装に直結する具体的な設計パターンと、将来的な拡張を見据えた技術的洞察を提供する。

我々が目指すべきは、エージェントが「恐らく正しい」コードを生成するのではなく、「仕様を満たすことが数学的に証明された」コードのみをシステムにコミットする世界の実現である。本稿が、その道標となることを確信する。

2. 動的オーケストレーションのパラダイムシフト: 静的グラフから進化的組織へ

マルチエージェントシステム(MAS)の設計思想は、2024年まで主流であった「人間が定義した静的な役割分担」から、2025年には「システム自体がタスクに応じて自己の構造を動的に書き換える進化的アプローチ」へと劇的な転換を遂げている。これは、AET-OSのような汎用OSにとって、未知のタスクや変動する環境に適応するために不可欠な進化である。

2.1 ADAS: コードによるエージェントの自動発明と自己増殖

この分野において最も革新的かつ理論的に深遠なアプローチの一つが、ADAS (Automated Design of Agentic Systems) である¹。従来のシステムでは、プロンプトエンジニアリングやファインチューニングによってエージェントの性能を向上させてきたが、ADASはこの前提を覆し、「メタエージェント」が新しいエージェントを実行可能なコード(主にPython)として定義・生成するという手法を採用している。

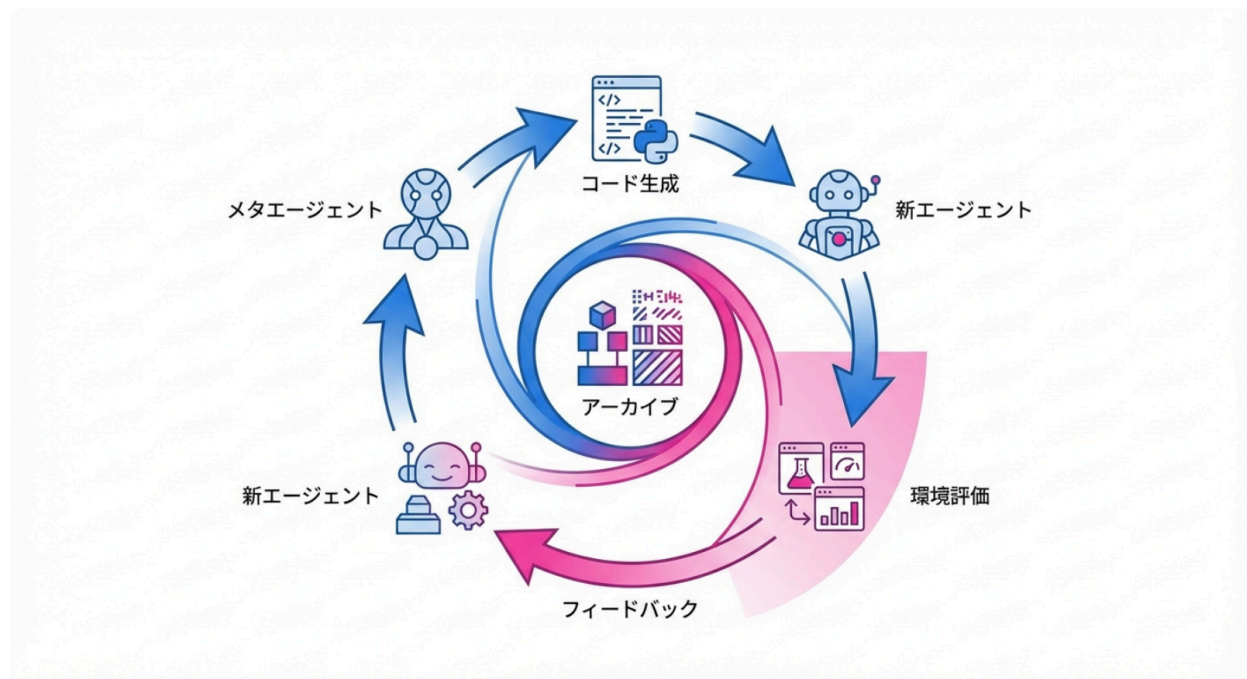
このアプローチの核心は、探索空間の質的な変化にある。プロンプトの探索空間は言語的制約を受けるが、チューリング完全なプログラミング言語(Pythonなど)を探索空間とすることで、メタエージェントは理論上、あらゆる可能なエージェントシステムを「発明」できることになる。例えば、スタックオーバーフローのような再帰的なエラーが発生した場合、従来の静的エージェントでは対処不能になることが多いが、ADASによって生成されたエージェントは、そのような状況を検知してデバッグ専用のサブエージェントを即座にインスタンス化し、メモリダンプを解析させるような新しい制御フローを自らコードとして記述し、実行することが可能となる。

ADASの中核アルゴリズムである「Meta Agent Search」は、以下の進化的なループを実行する：

1. **生成 (Generation):** メタエージェントが、現在のタスク要件(例:レガシーシステムのCOBOLコードをPythonにリファクタリングする)を分析し、それに特化した新しいエージェントのロジック、プロンプト、ツール利用定義を含むPythonコードを作成する。
2. **評価 (Evaluation):** 生成されたエージェントコードをサンドボックス環境で実行し、定義された評価関数(例えば、ユニットテストの通過率や実行速度、トークン消費量)に基づいてパフォーマンスを測定する。
3. **アーカイブと学習 (Archiving & Learning):** 成功したエージェントの設計パターンをアーカイブに追加する。メタエージェントはこのアーカイブを参照し、過去の成功例を組み合わせたり、変異させたりすることで、次世代のより強力なエージェントを生成する。

このプロセスは、生物学的進化における遺伝的アルゴリズム(GA)と類似しているが、変異と交叉がランダムではなく、LLMの高い推論能力によって「意図的かつ論理的」に行われる点が決定的に異なる。AET-OSにおいて、このメカニズムを導入することは、システムが稼働中に自己の機能を拡張し続けることを意味する。

ADAS: コード空間におけるエージェントの自己進化的探索



メタエージェントはタスク要件に基づき、新しいエージェントロジックをPythonコードとして生成する。生成されたエージェントは環境で評価され、優秀な設計パターンのみがアーカイブに保存され、次世代のエージェント生成に利用される。

2.2 PuppeteerパターンとAgentOrchestra: 動的リソース配分と階層化

エージェントの「生成」だけでなく、実行時の「制御」においても大きな進展が見られる。**Puppeteer** (人形使い) パターン⁶は、静的なルーターやマネージャーに代わる新しいオーケストレーション手法である。従来のMASでは、タスクのルーティングポリシー(どのタスクをどのエージェントに投げるか)は事前に決定されているか、単純なルールベースであることが多かった。これに対し、Puppeteerパターンでは、中央のオーケストレーター(Puppeteer)が強化学習(RL)によって訓練されており、タスクの刻々と変化する状態(State)を監視しながら、動的にエージェント(Puppet)を選択・起動・停止する。

特筆すべきは、Puppeteerが「エージェントの剪定(Pruning)」を行う能力を持つ点である。タスクが進行するにつれて、初期には必要だった探索的なエージェントが不要になったり、逆にコストの高い専門エージェントが必要になったりする。Puppeteerはこの変化を検知し、貢献度の低いエージェントを動的に切り離し、より効率的なエージェント構成へと「組織図」をリアルタイムで書き換える。これにより、トークン消費の削減と推論精度の向上を同時に達成している。

一方、**AgentOrchestra**¹⁰は、この動的制御を厳格な階層構造の中で実現するフレームワークである。「Conductor(指揮者)」が最上位で戦略的な計画立案を行い、「Supervisor」が特定のドメイン(例: データベース操作、UI生成)を管轄し、「Worker」が具体的な作業を実行するという3層構造を持

つ。AgentOrchestraの特徴は、各層の役割が明確に分離されており、上位層は下位層の具体的な実装を知る必要がないというカプセル化にある。これにより、AET-OSのような大規模システムにおいて、特定のエージェント群（例えば、Pythonのバージョンアップに伴うコード修正チーム）をモジュールとして容易に追加・置換することが可能となる。

2.3 Microsoft AutoGen v0.4: イベント駆動型アーキテクチャへの進化

これらの理論的な進歩を実装レベルで支えるのが、Microsoft Researchによる**AutoGen**の大規模アップデート(v0.4)である¹⁴。v0.4における最大の変更点は、従来の「会話ベース(Conversation-based)」の制御フローから、イベント駆動型(**Event-driven**)かつ非同期メッセージングを核としたアーキテクチャへの完全な移行である。

以前のバージョンでは、エージェント間のやり取りはチャット履歴として管理されており、複雑な条件分岐や並列処理を記述するのが困難であった。しかし、v0.4ではエージェントが明確な入力型定義と出力スキーマを持つ「コンポーネント」として扱われ、クラウドネイティブな分散システムと同様に、メッセージバスを介して非同期に通信を行う。これにより、システム設計者は「エージェントAが失敗イベントを発行したら、エージェントBとCを並列で起動し、両者の結果をエージェントDが集約する」といった複雑なワークフロー(Workflow Graph)を、Pythonや.NETのコードとして堅牢に記述できるようになった。これは、AET-OSが目指す「OSのような基盤システム」にとって、極めて親和性の高いアーキテクチャであると言える。

3. 自己修正とリフレクションの最先端メカニズム: Halt Problemへの挑戦

コード生成エージェントにおける最大の技術的障壁は、自身のエラーを認識できずに無限ループに陥る「Halt Problem(停止性問題)」の変種、あるいは誤った修正を繰り返して元のコードよりも品質を劣化させる「Degeneration(劣化)」である。2025年の研究は、単純な「Self-Refine(自己修正)」プロンプトの限界を指摘し、より高度な検証メカニズムと学習手法を提案している。

3.1 SCoRe: 自己生成データを用いた強化学習による修正能力の獲得

Google DeepMindが発表した**SCoRe (Self-Correction via Reinforcement Learning)**¹⁷は、エージェントの「自己過信(Overconfidence)」バイアスを打破するための画期的な手法である。

従来の教師あり微調整(SFT)では、エージェントは「一度で正解(Gold Standard)を出す」ように訓練される。そのため、推論時にエラーが発生して修正を求められても、モデルの重みは「自分の最初の出力は正しいはずだ」というバイアスに支配されており、表面的な修正(変数のリネームやコメントの変更など)しか行わない傾向が強かった。SCoReはこの問題に対し、モデル自身が生成した「誤った回答」と、そこからの「修正プロセス」のトレース(履歴)を用いて強化学習を行うことで対処する。

SCoReのトレーニングプロセスは以下の2段階で構成される:

1. **Stage 1 (Policy Initialization):** 初期の回答分布を大きく崩さないように制約をかけつつ、修正を行うための基本的なポリシーを初期化する。これにより、モデルが「修正すること」自体を学習し始める。

2. **Stage 2 (Reward Shaping):** 実際に「一度間違えてから、自力で正解に修正できた」ケースに対して、特に高い報酬を与える。単に正解することよりも、「リカバリーに成功すること」を高く評価する報酬設計により、モデルは失敗からの復旧能力を重点的に強化される。

AET-OSのエージェントにSCoReの手法を適用することで、「最初から完璧なコードを書く」ことよりも、「バグを含んだドラフトを書き、テスト結果や検証ツールのフィードバックを見て確実に修正する」能力を最大化できる。これは、実世界のソフトウェア開発プロセス（書いて、テストして、直す）の本質により近い挙動である。

3.2 SETS: 推論時のスケーリングと統合的検証

モデルの学習時だけでなく、推論時 (Test-time) の計算リソース配分を最適化する手法として、**SETS (Self-Enhanced Test-Time Scaling)**²¹ が提案されている。これまでの研究では、多数の候補を生成して最良のものを選ぶ「Best-of-N」サンプリングや、逐次的に修正を行う「Sequential Refinement」が個別に議論されてきたが、SETSはこれらを「サンプリング」「自己検証 (Self-Verify)」「自己修正 (Self-Correct)」という三つの操作として統合的なフレームワークに統合する。

SETSの重要な知見は、検証器 (Verifier) の役割である。AET-OSへの適用において特に重要なのは、SETSが示唆する「検証器の独立性」である。コードを生成したのと同じプロンプトやモデルで検証を行うと、生成時のバイアスが検証時にも影響し、ミスを見逃す可能性が高い。したがって、検証フェーズには、生成とは異なる視点（例えば、異なるプロンプト戦略、あるいは後述する形式手法のような外部ツール）を持つ独立したプロセスを割り当てる設計パターンが有効である。

3.3 Ralph Loop: 持続性と終了条件の工学的制御

エージェントが無限ループに陥るのを防ぐためのより実践的かつ工学的なアプローチとして、**Ralph Loop (Persistence Loop)**²⁵ という概念が提唱されている。これは、従来のReActループ (Plan-Execute) がLLMの自律的な判断のみに依存して終了条件を決定していたのに対し、外部からの「強制的な介入」と「持続性の管理」を明示的に組み込んだものである。

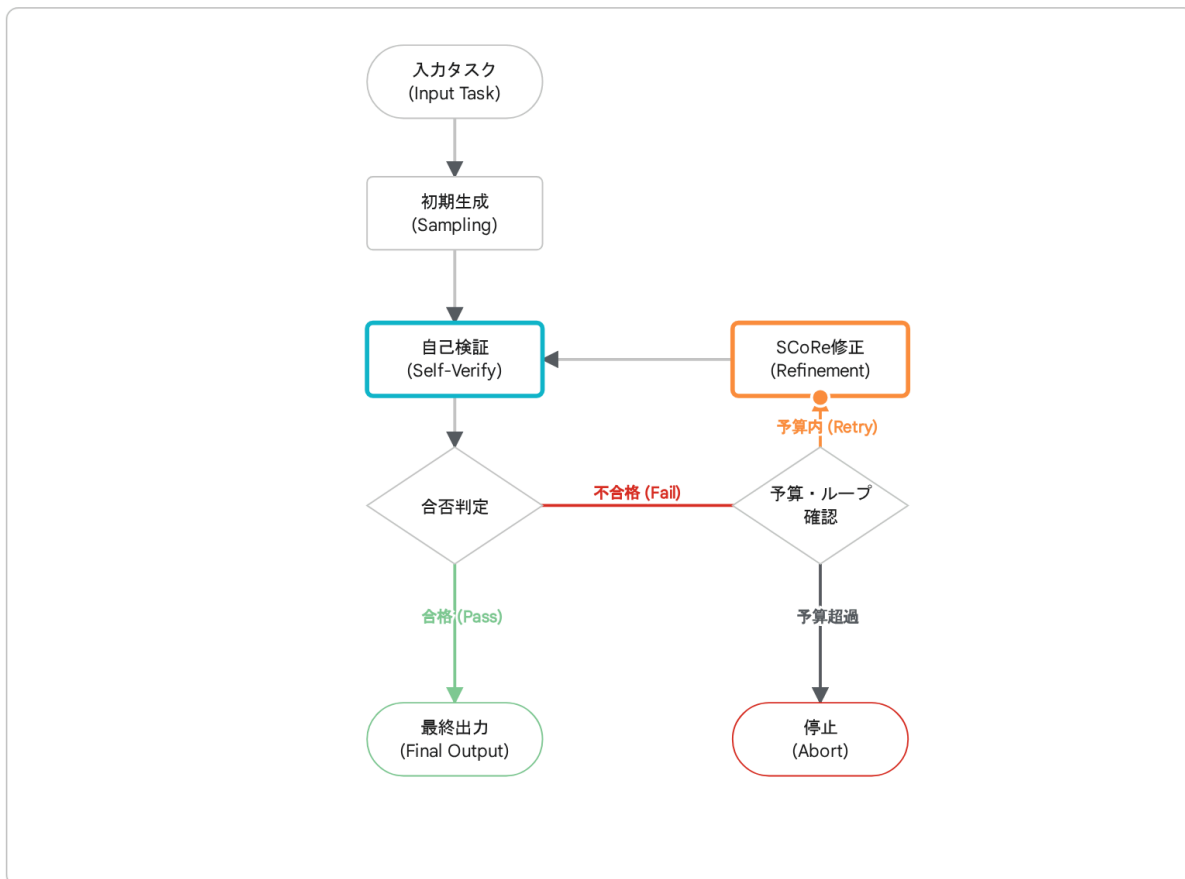
Ralph Loopでは、エージェントの状態を外部のデータベース (AET-OSの場合はLangGraphのCheckpoint機能などが該当) に永続化し、タスクの進捗が停滞した場合 (例: 同じエラーログが3回以上続いた場合、あるいは検証スコアの向上が見られない場合) に、外部の監視プロセス (Watchdog) が介入してエージェントを再起動したり、戦略を強制的に変更したりする。また、トークン予算や時間制限といったリソース制約を終了条件 (Halt Condition) として厳密に定義することで、エージェントの暴走を物理的に阻止する。

AET-OS向け自己修正パイプライン：SETSとSCoReの統合モデル

● 検証 (SETS Verify)

● 修正 (SCoRe Policy)

アニメーション再生



初期生成後、SETSフレームワークに基づく検証（Verify）が行われる。失敗時はSCoReで強化された修正ポリシー（Correct）が発動。無限ループ防止のため、Ralph Loopのような持続性制御とトークン予算管理が組み込まれている。

Data sources: [SETS Paper](#), [SCoRe Paper](#), [Ralph Loop Pattern](#)

4. 形式検証 (Formal Verification) の統合：AET-OSの核心技術

貴殿が最も重要視する「形式検証」の統合は、ニューラルネットワークの確率論的な限界を突破し、AET-OSに「証明可能な信頼性」をもたらすための核心技術である。2025年の研究において、ニューラルネットワーク (LLM) とシンボリックAI (Solver/Verifier) を融合させる**ニューロシンボリック (Neuro-symbolic)**アプローチは、実験室レベルを超えて実用的な実装段階に入りつつある。

4.1 VeCoGen: LLMによる形式仕様の自動生成とフィードバックループ

この分野における代表的なアーキテクチャの一つが、**VeCoGen (Verified Code Generation)**²⁶である。通常、形式検証の最大のボトルネックは、人間が数理的な仕様 (Specification) を記述するコストが高いことにある。VeCoGenはこの問題を、LLMにコードだけでなく仕様自体も生成させることで解決を図る。

VeCoGenのプロセスは以下の通りである：

1. 仕様生成: LLMが自然言語の要求 (例:「配列の要素をソートする」)を受け取り、対応するCコードと共に、ANSI/ISO C Specification Language (ACSL) 形式の注釈 (事前条件、事後条件、不変条件)を生成する。
2. 形式検証: 生成されたコードと仕様を**Frama-C**などの形式検証ツールに入力する。検証ツールは、WP (Weakest Precondition) プラグインなどを用いて、コードが仕様を論理的に満たすかどうかを証明しようとする。
3. フィードバック: 証明に失敗した場合、検証ツールは具体的な反例 (Counter-example) やエラー箇所を出力する。VeCoGenはこのエラーログをLLMにフィードバックし、LLMはそれをヒントにコードまたは仕様を修正する。

このアーキテクチャの強みは、LLMが苦手とする「厳密な論理整合性のチェック」を、Z3 Solverなどの外部ツールに完全に委譲できる点にある。LLMは「翻訳者」として自然言語を形式言語に変換し、検証ツールは「監査役」としてその正当性を保証する。AET-OSにおいても、システム設計書からTLA+やAlloyのコードを生成させ、それをモデル検査器にかけるフローを組み込むことで、設計段階での論理矛盾 (デッドロックや競合状態)を自動検出できる。

4.2 QWEDプロトコルとCrossHair: Pythonコードの数学的検証

より現代的かつPythonエコシステムに親和性の高いアプローチとして、**QWED**プロトコル²⁸ および **CrossHair**³⁰ の活用が挙げられる。

QWED (Query With Evidence and Determinism) は、「LLMは信頼できない翻訳者 (Untrusted Translator) である」という哲学に基づいている。LLMの役割は、自然言語のクエリを特定のドメイン固有言語 (DSL) やSQL、あるいは数式に変換することだけに限定される。その後の計算や論理判定は、**決定論的なソルバー (SymPy, Z3, SQLGlot) **が行う。これにより、エージェントの出力は「確信度80%」のような曖昧なものではなく、「証明済み (Proven)」か「検証不能 (Unverified)」かの明確な二値判定が可能になる。QWEDはLangChainやCrewAIといった既存のフレームワークに「検証レイヤー」としてプラグイン可能であり、AET-OSへの統合も容易である。

CrossHairは、Pythonコードに対して**シンボリック実行 (Symbolic Execution) **を行うツールである。これは、具体的な値 (テストケース) を使ってコードを実行するのではなく、変数を記号 (Symbol) として扱い、あらゆる可能な入力パターンに対してコードがクラッシュしないか、あるいは特定の条件 (契約) を満たすかを数学的に検証する。AET-OSの実装においては、エージェントにPythonの型ヒントとassert文 (契約プログラミング) を豊富に含んだコードを書かせ、バックグラウンドでCrossHairを実行することで、エッジケースにおけるバグを開発中に自動的にあぶり出すことが可能となる。例えば、`def divide(a: int, b: int) -> float:` という関数に対し、`pre: b != 0` という事前条件をLLMが生成し、

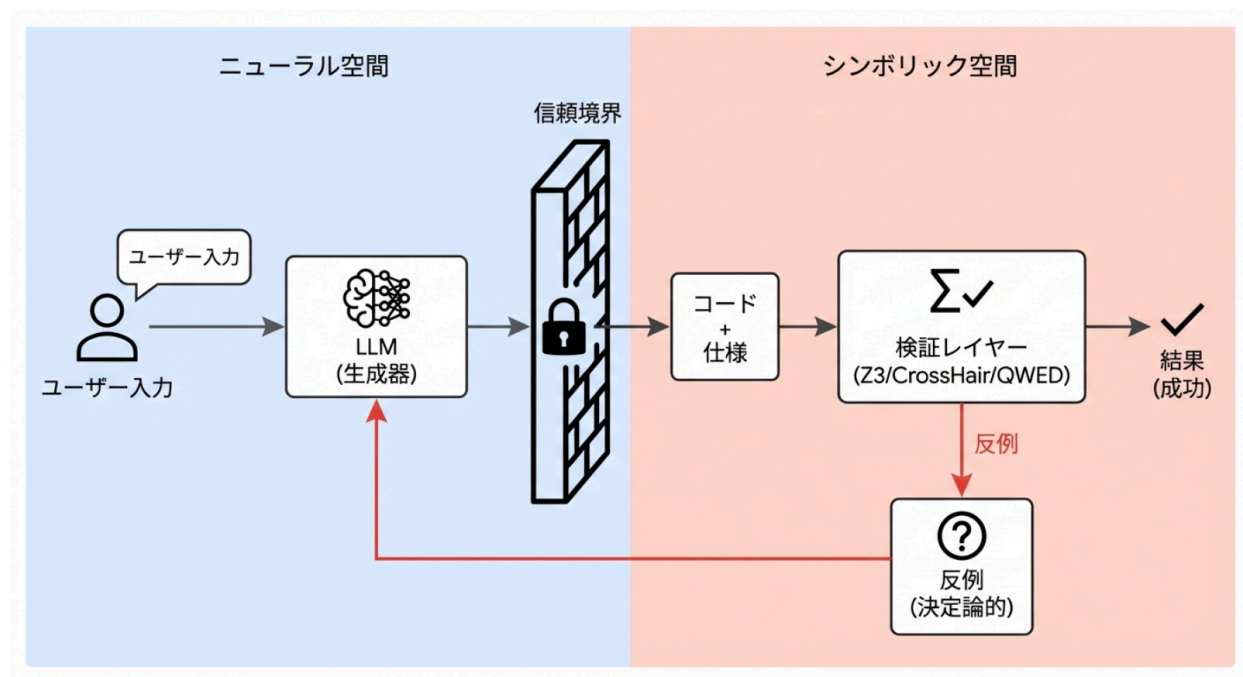
CrossHairがそれを検証することで、ゼロ除算の可能性を静的に排除できる。

4.3 Lean 4とDeepSeek-Prover-V2: 究極の信頼性へ

さらに高度な数学的証明が必要な領域（例えば、金融取引のコアロジックや暗号プロトコルの実装）では、証明支援言語**Lean 4**と、それに特化したモデル**DeepSeek-Prover-V2**³²の連携が注目されている。DeepSeek-Prover-V2は、自然言語の問題記述からLean 4の形式的な証明コードを生成することに特化しており、MiniF2FなどのベンチマークでSOTAを達成している。

AET-OSにおいて、特に高い信頼性が求められるモジュールについては、通常のPythonコードではなくLean 4による記述と証明をエージェントに行わせるという選択肢も、将来的には視野に入れるべきである。これは、コードが仕様を満たすことを数学的に「証明」するアプローチであり、バグの存在確率を限りなくゼロに近づけることができる。

Neuro-Symbolic Architecture: 信頼できない生成器と信頼できる検証器の統合



LLMは「翻訳者」として自然言語をコードおよび形式仕様（Contract/Spec）に変換する。これらは「信頼境界（Trust Boundary）」を超えて形式検証エンジン（Symbolic Engine）に渡される。Solver（Z3など）が反例（Counter-example）を発見した場合、それは決定論的なフィードバックとしてLLMに戻され、修正ループが回る。

5. AET-OS実装に向けた設計パターンとツール選定

以上の技術調査に基づき、AET-OSの実装において採用すべき具体的な設計パターンとツールセットを提案する。これらは、最新の研究成果を実用的なシステムへと落とし込むための青写真となる。

5.1 設計パターン: Verified Orchestrator Pattern

単一のエージェントにすべてを任せるのではなく、役割と権限を明確に分離した以下の3層構造を推奨する。このパターンは、AgentOrchestraの階層性とVeCoGenの検証ループを統合したものである。

層 (Layer)	役割 (Role)	責務 (Responsibilities)	推奨技術・パターン
1. 戦略層 (Strategic)	Meta-Architect	タスクの分解、リソース配分、サブエージェントの動的生成	ADAS / AgentOrchestra メタエージェントがタスクを分析し、必要な専門エージェントをコードとして生成・配備する。
2. 実行層 (Execution)	Worker / Specialist	コード生成、リファクタリング、設計記述の実装	Puppeteer Pattern タスクの状態に応じて、Python専門家やDB専門家などのエージェントを動的に切り替えながら作業を進める。
3. 検証層 (Verification)	Verifier / Auditor	形式仕様の生成、ソルバー実行、安全性チェック	VeCoGen / QWED / CrossHair 生成物に対して形式検証を行い、合格したもののみをコミットする。 **拒否権 (Veto Power) ** を持つ。

この構造において最も重要なのは、検証層が実行層に対して独立しており、かつ強い権限を持つこ

とである。実行層のエージェントがどれほど自信を持ってコードを提出しても、検証層のZ3 Solverが論理矛盾を検出した場合、そのコードは絶対に採用されず、修正要求と共に差し戻される。この「権力分立」こそが、確率的なLLMの暴走を防ぐ唯一の手段である。

5.2 実装ツールとフレームワークの選定

AET-OSの実装においては、以下のオープンソースソフトウェア(OSS)やライブラリの採用が推奨される。

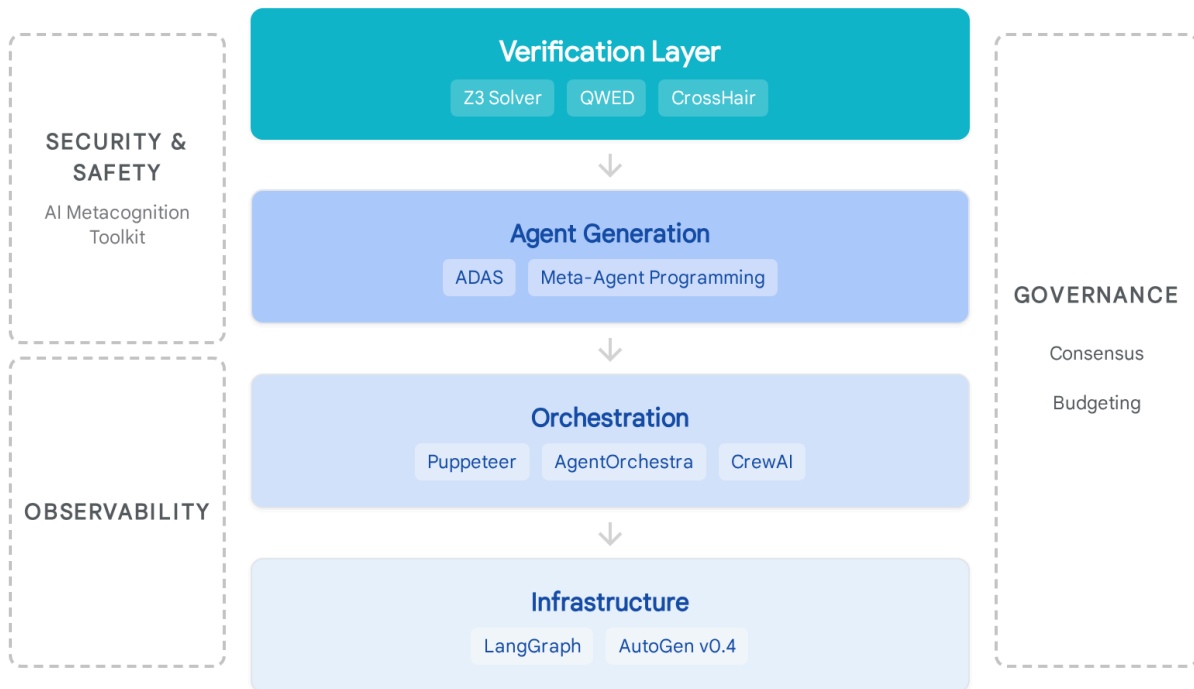
- **LangGraph:** ³⁴
 - 選定理由: AET-OSのバックボーンとして最適である。LangChainエコシステムの一部でありながら、循環グラフ(Cyclic Graph)をネイティブにサポートしており、SETSのような反復的な検証ループや、Puppeteerのような状態遷移をPythonコードで明示的に記述できる。また、チェックポイント機能(Persistence)により、Ralph Loopの実装(長時間走行時の状態保存と復帰)が容易である。
- **Microsoft AutoGen (v0.4):** ¹⁴
 - 選定理由: 複数のエージェント間の非同期通信基盤として利用する。特に、異なる言語(Pythonと.NETなど)で書かれたエージェントを連携させる場合や、イベント駆動型(Event-driven)の疎結合な設計にする場合に強力な威力を発揮する。
- **CrossHair / Z3 Prover:** ³⁰
 - 選定理由: Pythonコードの形式検証エンジンとして組み込む。エージェントにはassert文や型ヒントを豊富に含んだコードを書かせ、これをCrossHairで検証させるパイプラインを構築する。これにより、テストケースを書かずとも、数学的な網羅性を持ってコードの正しさを検証できる。
- **CrewAI + QWED:** ²⁸
 - 選定理由: 特定の役割(例:リサーチャー、ライター)を持つエージェントを素早く定義・配備する際に有用である。特にQWEDのCrewAI統合を利用すれば、わずか数行のコードで「検証機能付きエージェント(Verification-enabled Agent)」をシステムに追加できるため、開発の初期段階でのプロトタイピングに適している。

5.3 無限ループとHalt Problemへの対策: ConsensusとAegean-Serve

形式検証が適用できない、より曖昧なタスク(例: 自然言語による要約や、主観的な設計判断)において、エージェントが結論を出せずにループするのを防ぐために、**合意形成(Consensus)**のアプローチが有効である。

Aegean-Serve ³⁷ などのプロトコルは、複数の異なるモデル(例: GPT-4oとClaude 3.5 Sonnet、Gemini 1.5 Pro)に同じ問題を独立して解かせ、その結果が一定以上一致(Consensus)した場合にのみ、それを「正解」として採用し、プロセスを停止させる手法である。これにより、単一モデルの幻覚やバイアスによる暴走を防ぐと同時に、停止条件を「合意」という客観的な指標に置き換えることができる。AET-OSにおいては、重要な意思決定ポイントでこの合意形成プロトコルを発動させる設計が推奨される。

AET-OS: 推奨アーキテクチャスタック (2025)



LangGraphの状態管理を基盤とし、ADASによる動的エージェント生成、Puppeteerの実行制御、Neuro-symbolicな検証層（Z3等）を統合。右側にはコンセンサスと予算管理のガバナンス機能を配置。

Data sources: [LangChain](#), [Microsoft AutoGen](#), [VeCoGen/Z3](#), [CrewAI Projects](#), [ADAS Paper](#)

6. 結論: 確率から証明へ

本調査を通じて明らかになったのは、AIエージェントアーキテクチャが「確率的なテキスト生成の応用」というフェーズを終え、「工学的に制御された自律システム」という新たなフェーズに突入したという事実である。

AET-OSが目指すべき方向性は明確である。それは、ADASのような動的な適応能力で柔軟性を確保しつつ、VeCoGenやQWEDのような形式手法で出力を厳格にフィルタリングし、SCoReのような自己修正メカニズムで失敗から学び続けるアーキテクチャの構築である。AIの「創造性」と計算機科学の「厳密性」を分離し、パイプラインの中で再統合することこそが、2025年におけるState-of-the-Artの実装となるだろう。

推奨される第一歩は、LangGraphを用いて「生成エージェント」と「CrossHair検証エージェント」の最小構成ループ (Minimal Viable Loop) を構築し、そこにSCoReの考え方を取り入れて、失敗履歴からの自己修正能力をエージェントに学習させることである。この堅牢な核 (Kernel) の上に、動的なオー

ケストレーション機能を段階的に拡張していくことが、AET-OSの成功への最短経路であると確信する。

引用文献

1. AUTOMATED DESIGN OF AGENTIC SYSTEMS - ICLR Proceedings, 1月 21, 2026にアクセス、
https://proceedings.iclr.cc/paper_files/paper/2025/file/36b7acf6f6010652b3f2a433774a66fe-Paper-Conference.pdf
2. ShengranHu/ADAS: [ICLR 2025] Automated Design of Agentic Systems - GitHub, 1月 21, 2026にアクセス、<https://github.com/ShengranHu/ADAS>
3. Automated Design of Agentic Systems | OpenReview, 1月 21, 2026にアクセス、
<https://openreview.net/forum?id=t9U3LW7JVX>
4. ADAS - Shengran Hu, 1月 21, 2026にアクセス、
<https://www.shengranhu.com/ADAS/>
5. arXiv:2408.08435v2 [cs.AI] 2 Mar 2025, 1月 21, 2026にアクセス、
<https://arxiv.org/abs/2408.08435>
6. Beyond Rule-Based Workflows: An Information-Flow-Orchestrated Multi-Agents Paradigm via Agent-to-Agent Communication from CORAL - arXiv, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2601.09883v1>
7. Multi-Agent Collaboration via Evolving Orchestration - arXiv, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2505.19591v2>
8. Multi-Agent Collaboration via Evolving Orchestration - Sparrow Intelligence, 1月 21, 2026にアクセス、
<https://blog.sparrow.so/multi-agent-collaboration-via-evolving-orchestration-the-complete-guide-to-dynamic-ai-agent-systems/>
9. Multi-Agent Collaboration via Evolving Orchestration - arXiv, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2505.19591v1>
10. AgentOrchestra: A Hierarchical Multi-Agent Framework for General-Purpose Task Solving, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2506.12508v2>
11. AgentOrchestra: A Hierarchical Multi-Agent Framework for General-Purpose Task Solving, 1月 21, 2026にアクセス、
<https://skyworkai.github.io/DeepResearchAgent/>
12. AgentOrchestra: A Hierarchical Multi-Agent Framework for General-Purpose Task Solving, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2506.12508v1>
13. AgentOrchestra Explained: A Mental Model for Hierarchical Multi-Agent Systems - Medium, 1月 21, 2026にアクセス、
<https://medium.com/@phoenixarjun007/agentorchestra-explained-a-mental-model-for-hierarchical-multi-agent-systems-c8985d083115>
14. AutoGen - Microsoft Research: News And Awards, 1月 21, 2026にアクセス、
<https://www.microsoft.com/en-us/research/project/autogen/news-and-awards/>
15. AutoGen - Microsoft Research, 1月 21, 2026にアクセス、
<https://www.microsoft.com/en-us/research/project/autogen/>
16. AutoGen to Microsoft Agent Framework Migration Guide | Microsoft ..., 1月 21, 2026にアクセス、

- <https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-autogen/>
17. TRAINING LANGUAGE MODELS TO SELF-CORRECT VIA REINFORCEMENT LEARNING - ICLR Proceedings, 1月 21, 2026にアクセス、
https://proceedings.iclr.cc/paper_files/paper/2025/file/871ac99fdc5282d0301934d23945ebaa-Paper-Conference.pdf
 18. Google Publishes LLM Self-Correction Algorithm SCoRe - InfoQ, 1月 21, 2026にアクセス、<https://www.infoq.com/news/2024/10/google-deepmind-score/>
 19. Training Language Models to Self-Correction via Reinforcement Learning: A Deep Dive into SCoRe with Code Implementation using PyTorch. | by Devmallya Karar | Medium, 1月 21, 2026にアクセス、
<https://medium.com/@devmallyakarar/training-language-models-to-self-correct-on-via-reinforcement-learning-a-deep-dive-into-score-with-ff85421b4186>
 20. [2409.12917] Training Language Models to Self-Correct via Reinforcement Learning - arXiv, 1月 21, 2026にアクセス、<https://arxiv.org/abs/2409.12917>
 21. SETS: Leveraging Self-Verification and Self-Correction for Improved Test-Time Scaling, 1月 21, 2026にアクセス、<https://arxiv.org/html/2501.19306v3>
 22. Self-Correcting Code Generation Using Small Language Models - ACL Anthology, 1月 21, 2026にアクセス、<https://aclanthology.org/2025.findings-emnlp.127.pdf>
 23. Self-Correcting Code Generation Using Small Language Models - arXiv, 1月 21, 2026にアクセス、<https://arxiv.org/html/2505.23060v3>
 24. SETS: Leveraging Self-Verification and Self-Correction for Improved Test-Time Scaling - OpenReview, 1月 21, 2026にアクセス、
<https://openreview.net/pdf?id=Wv9NMJoKww>
 25. From ReAct to Ralph Loop A Continuous Iteration Paradigm for AI Agents - Alibaba Cloud, 1月 21, 2026にアクセス、
https://www.alibabacloud.com/blog/from-react-to-ralph-loop-a-continuous-iteration-paradigm-for-ai-agents_602799
 26. VeCoGen: Automating Generation of Formally Verified C Code with Large Language Models - arXiv, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2411.19275v3>
 27. Effectiveness of Large Language Models to Generate Formally Verified C code - kth .diva, 1月 21, 2026にアクセス、
<https://kth.diva-portal.org/smash/get/diva2:1915138/FULLTEXT02.pdf>
 28. projects | Subhadip Mitra, 1月 21, 2026にアクセス、
<https://subhadipmitra.com/projects/>
 29. QWED-AI/qwed-verification: Deterministic verification layer ... - GitHub, 1月 21, 2026にアクセス、<https://github.com/QWED-AI/qwed-verification>
 30. Enhancing LLM Code Generation with Ensembles: A Similarity-Based Selection Approach - arXiv, 1月 21, 2026にアクセス、<https://arxiv.org/html/2503.15838v1>
 31. (PDF) Python Symbolic Execution with LLM-powered Code Generation - ResearchGate, 1月 21, 2026にアクセス、
https://www.researchgate.net/publication/384075953_Python_Symbolic_Execution_with_LLM-powered_Code_Generation
 32. DeepSeek-Prover-V2: Open-Source AI for Lean 4 Formal Theorem Proving -

- Medium, 1月 21, 2026にアクセス、
<https://medium.com/aimonks/deepseek-prover-v2-open-source-ai-for-lean-4-formal-theorem-proving-ab7f9910576b>
33. CLEVER: A Curated Benchmark for Formally Verified Code Generation - arXiv, 1月 21, 2026にアクセス、<https://arxiv.org/pdf/2505.13938>
 34. LangGraph Systems Inspector: An AI Agent for Testing and Verifying LangGraph Agents | by Nirdiamant | Medium, 1月 21, 2026にアクセス、
<https://medium.com/@nirdiamant21/langgraph-systems-inspector-an-ai-agent-for-testing-and-verifying-langgraph-agents-a8d1c2400d60>
 35. LangGraph - LangChain, 1月 21, 2026にアクセス、
<https://www.langchain.com/langgraph>
 36. @qwed-ai/sdk 1.0.0 on npm - Libraries.io - security & maintenance data for open source software, 1月 21, 2026にアクセス、
<https://libraries.io/npm/@qwed-ai%2Fsdk>
 37. Reaching Agreement Among Reasoning LLM Agents - arXiv, 1月 21, 2026にアクセス、
<https://arxiv.org/html/2512.20184>